

To this end, we switch to another notion of information based on compression, then later we show how this notion closely approximates the notion of memorization defined above. Kolmogorov complexity defines the information content of a string x , denoted as $H^K(x)$, to be the length of shortest representation of x in a given computational model. Similarly, we can define the leftover information $x \mid \theta$, to be the shortest representation of x , when we have θ available as a reference. And the information content of $x \mid \theta$, denoted by $H^K(x \mid \theta)$, is the length of such description. Then, we can define mutual information in a similar fashion:

Definition 2 (Kolmogorov complexity). Let f be an arbitrary computational model that takes a set of inputs and returns an output (e.g. universal Turing machine). The shortest description of x with respect to computational model f is defined as $H^K(x) = \min_{f(p)=x} |p|$. Also, the Kolmogorov complexity of x relative to another string θ is defined as $H^K(x \mid \theta) = \min_{f(p,\theta)=x} |p|$. And we define the Kolmogorov mutual information between x and θ by $I^K(x, \theta) = H^K(x) - H^K(x \mid \theta)$. We assume inputs are bitstrings and $|p|$ is the bit length of the input.

Definition 3 (Kolmogorov memorization). Let θ be a reference model that approximates the true distribution of data, and $\hat{\theta}$ be a model trained on a dataset $x = (x_1, \dots, x_n)$. For each x_i we define the memorization of x_i in $\hat{\theta}$ as $\text{mem}^K(\hat{\theta}, x) = I^K(\hat{\theta}, x)$. We also define intended and unintended variants of memorization:

$$\text{mem}_U^K(x, \theta, \hat{\theta}) = H^K(x \mid \theta) - H^K(x \mid (\theta, \hat{\theta})), \text{ and } \text{mem}_I^K(x, \theta, \hat{\theta}) = \text{mem}^K(x, \hat{\theta}) - \text{mem}_U^K(x, \theta, \hat{\theta}).$$

There are known connections between Kolmogorov complexity and Shannon Entropy (Grunwald & Vityanyi, 2004). These results point at the conceptual connection between the two notions and imply that $\mathbb{E}_{x \sim X}[H^K(x)] \approx H(X)$. Interestingly, this implies that our notion of Kolmogorov memorization closely approximates Shannon memorization.

Proposition 4. Let $X = (X_1, \dots, X_n)$ be an i.i.d, dataset distribution parametrized by ground-truth model θ . Let L be a training algorithm mapping X to $\hat{\theta}$. Assume $H(\hat{\theta}) = \ell$ and $H(X_i) = \ell'$ ². Then we have $\left| \mathbb{E}_{\substack{x \sim X \\ \hat{\theta} \sim L(x)}} [\text{mem}_U^K(x_i, \hat{\theta}, \theta)] - \text{mem}_U^K(X_i, \hat{\theta}, \theta) \right| \leq \epsilon$. for some constant ϵ independent of ℓ, ℓ' and n .

2.3 Estimating Kolmogorov with compression

Fixing our notion of Kolmogorov memorization, we now describe how we can estimate H^K in different setups. Note that exact calculation of Kolmogorov complexity is known to be uncomputable (Kolmogorov, 1965). However, we can still approximate it using the best available compression schemes. These compression schemes could be arbitrary algorithms, e.g. using prompt optimization as in Schwarzschild et al. (2024) or using text prefixes as in Carlini et al. (2023b).

We adopt arithmetic coding as the most natural compression algorithm for language. Arithmetic coding is not only effective for text compression Delétang et al. (2024), but it also allows code lengths to be computed efficiently using model likelihoods. A promising direction for future research is to design compression algorithms specifically tailored to minimize the code length of training data in machine learning models and use it to obtain more accurate estimation of Kolmogorov complexity and memorization. Below, we summarize how we approximate each term in our memorization definition using model likelihoods.

- $H^K(x \mid \hat{\theta})$: Here, $\hat{\theta}$ is the trained target model, which does not necessarily capture the true data distribution. We do not really calculate the compressed code, instead we use fact that the compression rate of arithmetic coding tied to the likelihood of the model (Shannon, 1950). So, we can estimate $H^K(x \mid \hat{\theta})$ by the negative log likelihood of x under the target model, $-\log(p(x \mid \hat{\theta}))$.
- $H^K(x \mid \hat{\theta}, \theta)$: In this case, the compression algorithm has access to both target and reference models. We simply compute $-\log(\max\{p(x \mid \hat{\theta}), p(x \mid \theta)\})$. In practice, our choice of reference model is a larger model with the same architecture as θ trained for many steps on a much wider data distribution.

A curious reader may notice that we began with a likelihood-based notion of memorization, then shifted to a definition grounded in Kolmogorov complexity, and ultimately returned to likelihood to estimate that

²The trained model and each data sample can be presented using ℓ and ℓ' bits respectively.

n_{layer}	d_{model}	Params	Capacity(θ) [bits]		α [bpp]	
			fp32	bf16	fp32	bf16
1	32	8.04×10^4	3.39×10^5	3.16×10^5	4.23	3.93
	64	1.85×10^5	7.27×10^5	6.93×10^5	3.92	3.74
	128	4.69×10^5	1.71×10^6	1.69×10^6	3.65	3.61
	256	1.33×10^6	4.15×10^6	3.83×10^6	3.12	2.88
2	32	9.31×10^4	3.87×10^5	3.31×10^5	4.16	3.56
	64	2.35×10^5	9.60×10^5	9.27×10^5	4.08	3.94
	128	6.67×10^5	2.66×10^6	2.60×10^6	3.99	3.89
	256	2.12×10^6	8.49×10^6	7.76×10^6	4.01	3.66
4	32	1.18×10^5	4.65×10^5	3.99×10^5	3.92	3.37
	64	3.35×10^5	1.34×10^6	1.14×10^6	3.98	3.39
	128	1.06×10^6	4.02×10^6	3.75×10^6	3.78	3.53
	256	3.70×10^6	1.36×10^7	1.30×10^7	3.68	3.51
8	32	1.69×10^5	5.12×10^5	4.85×10^5	3.02	2.86
	64	5.35×10^5	2.05×10^6	1.71×10^6	3.83	3.19
	128	1.86×10^6	7.23×10^6	6.49×10^6	3.89	3.49
	256	6.86×10^6	2.71×10^7	2.51×10^7	3.96	3.65
					Mean (± 0.1):	
					3.83 3.51	

Table 1 Model capacity estimates across different widths and depths in full and half-precision. Doubling precision from bfloat16 to float32 only increases model capacity from 3.51 to 3.83 bits-per-parameter.

complexity. We emphasize, however, that the likelihood used in our approximation of Kolmogorov memorization is distinct from the initial likelihood notion. In particular, this likelihood is dependent on the parameters of the decoding algorithm, such as temperature or top-k sampling. More broadly, we note that any compression algorithm can be used to approximate Kolmogorov complexity, and our choice of arithmetic coding is just one instantiation of this broader framework.

Choice of reference model. In this work we use two reference models to compute $p(x | \hat{\theta})$. In our experiments on synthetic random strings (Section 3) we know the exact underlying data distribution and use that as a reference model. In our experiments on text (Section 4) we select θ to be a large model of the same family, trained on a much larger superset of the training data for $\hat{\theta}$.

3 Model Capacity for Memorization

Unintended memorization provides us a principled way of measuring the precise number of bits a model θ knows about a datapoint x . If we add up the information for each datapoint in a dataset, we can measure the total amount of bits a model knows about the dataset. And in cases where generalization is not possible because each datapoint is completely independent, we can estimate the **capacity** of a given model θ by summing per-datapoint unintended memorization.

3.1 Defining model capacity

We first formalize this notion of memorization capacity for a particular language model θ . Capacity is the total amount of memorization that can be stored in θ across all its parameters.

Definition 5 (Capacity). Let X be a distribution and $L: X \rightarrow \Theta$ a learning algorithm. We define the capacity

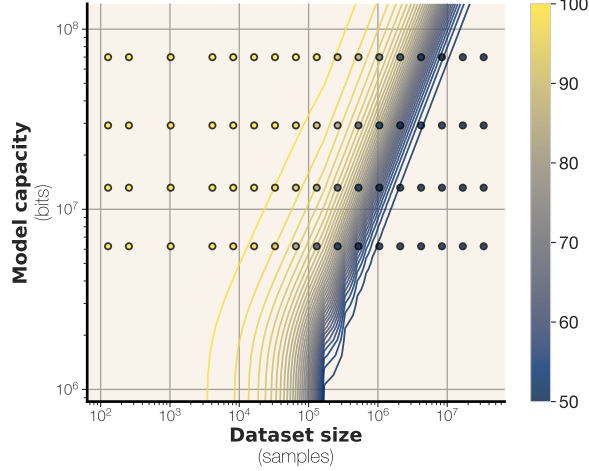


Figure 7 Scaling law curves for membership inference overlaid with empirical data, shown in circles.

of the learning algorithm L to be

$$\text{Capacity}(L) = \max_X \text{mem}(X, L(X))$$

When the model capacity is reached, $\text{mem}(X, L(X))$ will no longer increase with dataset size. In practice, we can compute capacity by training to saturation on varying sizes of X and computing the maximum memorization.

3.2 Measuring model capacity with synthetic sequences

In this section we measure the capacity of Transformer language models. Our goal is to instantiate multiple datasets and distributions and measure the memorization of them when training a single model θ . Then, we take the maximum over all datasets to approximate of the model’s capacity. For instantiating our datasets, each token is uniformly sampled from a predefined set of tokens independent of the previous tokens.

To approximate $H^k(x \mid \theta, \hat{\theta})$, we can directly compute entropy under the trained model to calculate the shortest description of the dataset conditioning on $\hat{\theta}$. Subtracting the two, we can approximate the unintended memorization $\text{mem}_U(X, L(X))$. Since the process for sampling the data is completely random, there is no generalization to be stored within $\hat{\theta}$ (that is, $\text{mem}^U(X, L(X)) \approx \text{mem}(X, L(X))$).

Observe that when we sample synthetic sequences from a uniform distribution, we can compute their Shannon information exactly. Given a dataset size N , we construct a dataset of N sequences, each of S tokens. Given a vocabulary size V , we can calculate the total entropy of a dataset x^i with such parameters by $H(x^i) = NS \log_2 V$. Then we calculate the compressed form x^i using entropy under $\hat{\theta}_i$ to compute the code length and use this as an approximation of $H^K(x^i \mid \hat{\theta}_j)$. Then we calculate the $\text{mem}(x^i, \hat{\theta}_i) = H(x^i) - H^K(x^i \mid \hat{\theta}_j)$ and compute a model’s capacity as the maximum amount of memorization over all datasets.

Experimental details. In accordance with [Kaplan et al. \(2020\)](#), we train models with the GPT-2 architecture ([Radford et al., 2019](#)) initialized from scratch. Our models have between 1 and 8 layers, hidden dimensions scaled from 32 to 512, and from 100K to 20M parameters. We train models for 10^6 steps with a batch size of 2048. We use the Adam optimizer. All models are trained on a single A100 GPU in bfloat16 precision, and we use gradient accumulation if a batch cannot fit in memory. Unless otherwise noted, we set vocabulary size $V = 2048$, sequence length $S = 64$ and vary only the number of points in a dataset. We train each model on each dataset size over five random seeds, which affect both model initialization and the dataset sampling.

Results. We plot memorization across model and data sizes in Figure 1. This allows us to visualize unintended memorization amounts (y-axis) across dataset sizes (x-axis) grouped by model size (line color). We observe a striking plateau once a model reaches its capacity. Given the dataset is large enough, models exhibit an upper bound in net memorization, regardless of data size. Small datasets are completely memorized by all models with enough capacity.

We estimate the capacity of each model as the maximum amount of unintended memorization in bits measured across all dataset sizes. We then compare this capacity to the model size in Figure 6. Interestingly, even at this small scale, we see a very smooth relationship between observed capacity (maximum memorization measured over all datasets) and model parameters. We plot this relationship in Figure 6: under these settings, our models consistently memorize between 3.5 and 3.6 bits per parameter. This corroborates the findings of prior work such as (Roberts et al., 2020; Lu et al., 2024), which noticed that fact storage scales linearly with model capacity. Ours is a slightly larger estimate than Allen-Zhu & Li (2024), which estimated via quantization that models can store around 2 bits per parameter.

Since our models are learned via gradient descent, they are not guaranteed to find the global optima; thus, we are only ever measuring a lower bound on model capacity. We take a closer look at the training curves to analyze the convergence of our 8M parameter language model. We plot model convergence throughout training in Figure 5.

In this case, all datasets from 16,000 to 4M samples fall within a range of $3.56 - 3.65 \times 10^6$ bits memorized. This indicates that our measurements are robust within an order of magnitude, and we do not expect to memorize significantly more information by training for more steps. This finding also confirms our hypothesis that capacity scales roughly with parameter count. The two largest datasets (4M and 8M samples, respectively) converge to total memorization of 2.95×10^6 and 1.98×10^6 bits memorized. We expect that their memorization rates would continue to increase toward the capacity had we trained for more epochs.

How does precision affect capacity? One natural question is how our estimates for α depend on the precision of language model training. In fact, although most software defaults to training in 32-bit precision, recent work has shown that language models can be quantized to fewer than 2 bits per parameter and still retain much of their utility. Since all other experiments have been conducted in bfloat16 precision, rerun our experiments in full fp32 precision to analyze the effect on capacity. Across model sizes, we observe a small increase in capacity, and an increase in α from 3.51 to 3.83 bits-per-parameter on average. This is far less than the actual 2x increase in the bits of θ , indicating that **most of the extra model bits added when increasing precision from bfloat16 to float32 are not used** for raw storage.

4 Disentangling Unintended Memorization from Generalization

Our previous experiments analyzed the memorization and membership inference properties of synthetic bitstrings. We now turn to measuring memorization of text. Unlike randomly generated sequences, learning from text data is a mix of both unintended memorization (sample-level) and generalization (population-level). Therefore, as a reference model, we use the model of an equal parameter count trained on the maximum amount of data (in this case, the entire dataset).³ We also consider an *oracle* reference model, which is the model that achieves the best compression rate (lowest loss) on the evaluation dataset, and may have many more parameters.

Experimental details. We repeat the experiments from Section 3.2, substituting our synthetic datapoints for real text. To obtain a distribution of real-world text data, we could use any pre-training scale text dataset; we use the recently proposed FineWeb dataset (Penedo et al., 2024) as it follows state-of-the-art deduplication practices. We use sequences of 64 tokens but perform an additional deduplication step to ensure perfect deduplication (otherwise, that 1 – 2% of sequences become duplicates when truncating to 64 tokens). We find careful deduplication extremely important for faithfully measuring extraction rates. As in the previous subsection, we pretrain models of varying sizes on different-sized text datasets and measure the unintended

³Restricting the computational power of the reference model relates its predictions to $\mathcal{V}$ -information (Xu et al., 2020) which measures the “usable” information available in a signal, when accounting for model size.